

PROJECT REPORT

CarePulse: Hospital Management System

Group No. 3

Subject: Java Programming

Technology Stack: Java SE 11+, Swing GUI

1. Abstract

The CarePulse Hospital Management System is a comprehensive desktop application developed in Java. It aims to digitize and streamline the administrative and clinical operations of a modern hospital. By replacing traditional, error-prone paper-based systems, CarePulse ensures that patient registrations, doctor directories, appointment scheduling, medical records, and billing are handled with maximal efficiency.

A primary academic objective of this project is to implement robust logic using core Java Data Structures and Object-Oriented Programming (OOP) paradigms entirely from scratch, without the use of external database management systems (DBMS) or third-party libraries. This report details the architectural design, algorithmic choices, and the specific application of HashMaps, TreeMaps, LinkedLists, and Arrays within the system.

2. Introduction

In the modern healthcare ecosystem, the speed and accuracy of data retrieval can directly impact patient outcomes. CarePulse was conceptualized to solve the latency issues observed in paper-based record keeping.

Key Objectives:

- Develop a responsive, user-friendly Graphical User Interface (GUI) using Java Swing.
- Guarantee $O(1)$ time complexity for critical operations like patient lookup.
- Maintain an automatically sorted patient directory for easy auditing.
- Ensure thread-safe data mutations using the Singleton design pattern.

3. System Architecture (MVC)

CarePulse is built on a strictly segregated 3-Layer architecture, analogous to the Model-View-Controller (MVC) design pattern. This ensures separation of concerns, making the codebase highly maintainable and scalable.

3.1 UI Layer (View)

Located in the `hospital.ui` package, this layer is composed of Java Swing components (`JFrame``, `JPanel``, `CardLayout``). It is exclusively responsible for rendering screens and capturing user input. The UI layer contains zero business logic and cannot modify data directly.

3.2 Service Layer (Controller)

Located in the `hospital.service` package. The `HospitalService` class acts as the central brain. It receives requests from the UI layer, processes the logic, and modifies the underlying data structures. It is implemented as a Thread-Safe Singleton.

3.3 Model Layer (Model)

Located in the `hospital.model` package. These are Plain Old Java Objects (POJOs) like `Patient`, `Doctor`, and `Appointment`. They encapsulate state and expose strict getter/setter methods to ensure data integrity.

4. Data Structures Implementation

To optimize performance without a database, specific data structures were chosen based on the mathematical time complexity of their primary operations.

4.1 HashMap (O(1) Access)

Use Case: Patient & Doctor Directories. **Implementation:** `HashMap<String, Patient>`.

HashMaps use a hashing function to compute an index into an array of buckets or slots, from which the desired value can be found. In CarePulse, whenever a receptionist searches for a Patient ID (e.g., 'PAT-1001'), the system retrieves the full patient record in O(1) constant time, eliminating the need to iterate through lists.

4.2 TreeMap (O(log n) Sorting)

Use Case: Alphabetical Patient Sorting. **Implementation:** `TreeMap<String, Patient>`.

A TreeMap is a Red-Black tree based NavigableMap implementation. It is sorted according to the natural ordering of its keys. Every time a new patient is registered in CarePulse, they are simultaneously added to the TreeMap. This guarantees that whenever the 'Sorted Records' panel is opened, the data is already sorted alphabetically, taking O(log n) time per insertion.

4.3 LinkedList (O(1) Insertion/Deletion)

Use Case: Appointment Queues & Medical History. **Implementation:** `LinkedList<Appointment>`.

Appointments operate on a First-In-First-Out (FIFO) basis. A LinkedList is ideal for queues because inserting a new appointment at the tail, or removing a completed appointment from the head, takes exactly O(1) time. Arrays would require shifting all elements, causing O(n) overhead.

4.4 Fixed Arrays

Use Case: Circular Buffer for Recent Patients. **Implementation:** `String[] recentPatients = new String[5];`.

Arrays are used for data with a strictly known maximum size. The Dashboard features a 'Recent Patients' view that only shows the last 5 registered IDs. Using an array with a modulo operator index provides a lightweight, highly efficient circular buffer.

5. Object-Oriented Principles

5.1 Singleton Pattern

The Singleton pattern ensures a class has only one instance, and provides a global point of access to it. If multiple instances of HospitalService were created, different UI panels would read from different, desynchronized Data Structures.

```
public class HospitalService {
    private static HospitalService instance;

    // Private constructor prevents external instantiation
    private HospitalService() { }

    // Synchronized ensures Thread Safety in multi-threaded environments
    public static synchronized HospitalService getInstance() {
        if (instance == null) {
            instance = new HospitalService();
        }
        return instance;
    }
}
```

5.2 Encapsulation

All model classes enforce encapsulation. Variables are declared private and can only be modified via public setter methods. This allows the system to implement validation (e.g., ensuring Age cannot be set to a negative number) before the data is mutated.

5.3 Comparable Interface (Polymorphism)

To allow the TreeMap to automatically sort the Patient objects, the Patient class implements the Comparable<Patient> interface and overrides the compareTo method.

```
public class Patient implements Comparable<Patient> {
    private String fullName;

    @Override
    public int compareTo(Patient other) {
        return this.fullName.compareToIgnoreCase(other.getFullName());
    }
}
```

6. Module Descriptions & User Interface

The application is divided into 7 distinct modules accessible via a side navigation menu powered by Java Swing's CardLayout.

6.1 Dashboard Module

The Dashboard provides real-time analytics by querying the sizes of the HashMaps and LinkedLists. It displays Key Performance Indicators (KPIs) such as total registered patients, total scheduled appointments, and pending invoice amounts.

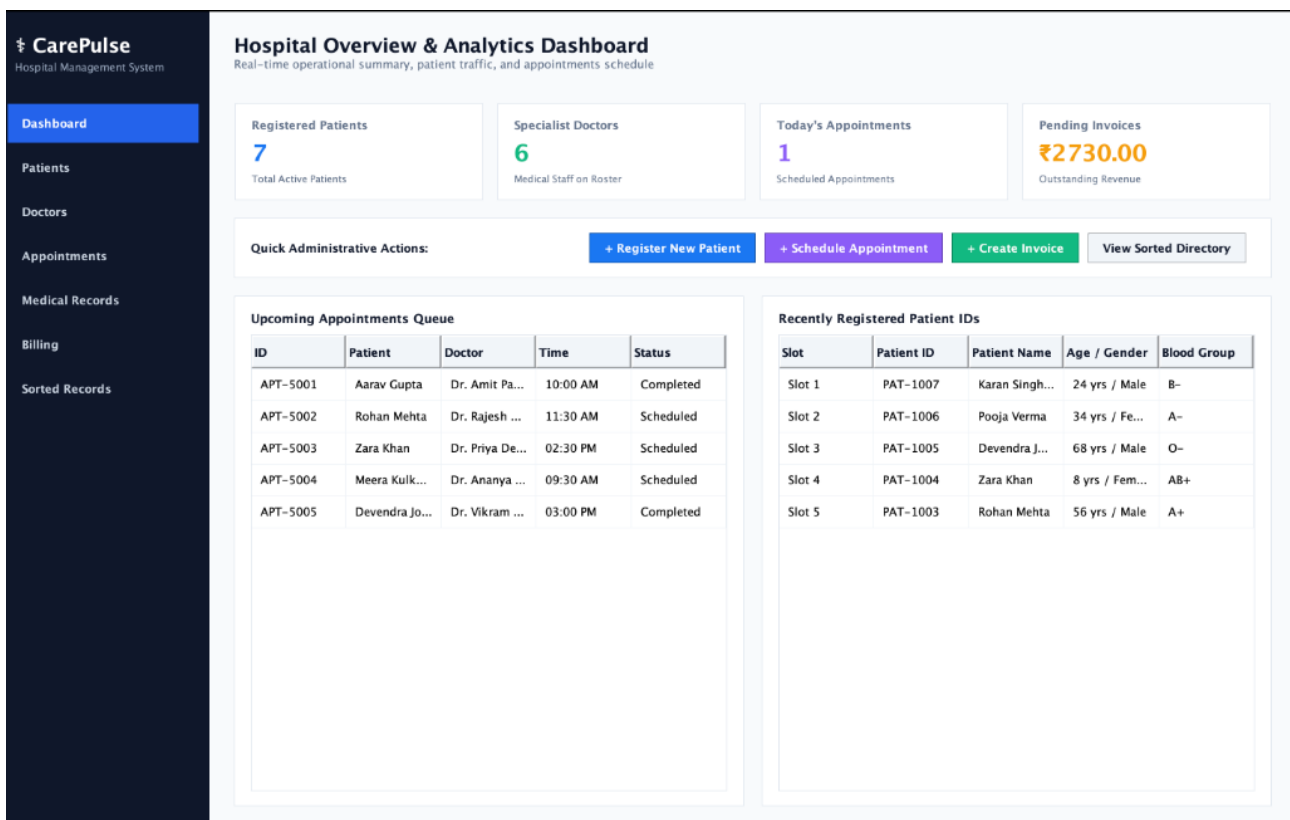


Figure 1: The Main Dashboard Interface

6.2 Patient Registration Module

This module features a comprehensive form to register new patient demographics. Upon submission, the data is packaged into a Patient object and injected into both the HashMap and TreeMap. The right panel allows for instant $O(1)$ searching by ID.

CarePulse
Hospital Management System

Dashboard

Patients

Doctors

Appointments

Medical Records

Billing

Sorted Records

Patient Registration & Records Management

Maintain patient demographics, medical identifiers, and fast HashMap search

Patient Details Form

Patient ID:

PAT-1001

Full Name *:

Age *:

Gender *:

Male

Blood Group:

A+

Contact Phone *:

Emergency Contact:

Blood Pressure:

120/80

Known Allergies:

None

Address:

Clear / New

Register Patient

Fast Search:

Search

Show All

View Full Profile

Delete Selected

Patient ID	Name	Age	Gender	Blood Group	Contact	Reg. Date	BP
PAT-1001	Aarav Gu...	29	Male	O+	+91-912...	2026-09...	120/80
PAT-1002	Meera Ku...	42	Female	B+	+91-912...	2026-09...	130/85
PAT-1003	Rohan M...	56	Male	A+	+91-912...	2026-09...	140/90
PAT-1004	Zara Khan	8	Female	AB+	+91-912...	2026-09...	110/70
PAT-1005	Devendra...	68	Male	O-	+91-912...	2026-09...	135/88
PAT-1006	Pooja Ver...	34	Female	A-	+91-912...	2026-09...	118/78
PAT-1007	Karan Sin...	24	Male	B-	+91-912...	2026-09...	122/80

Figure 2: Patient Registration and Search Panel

6.3 Doctor Directory Module

This module manages the medical staff. It allows hospital administrators to add new doctors and filter existing staff based on their specialization (e.g., Cardiology, Neurology).

CarePulse

Hospital Management System

Dashboard

Patients

Doctors

Appointments

Medical Records

Billing

Sorted Records

Medical Staff & Specialist Doctor Directory

Manage hospital physicians, clinical specialties, consultation schedules, and fees

Filter by Specialization: All

+ Add New Specialist

Doctor ID	Doctor Name	Specialization	Qualification	Consultation Fee (₹)	Room No	Available Schedule	Contact Phone
DOC-205	Dr. Amit Patel	General Medicine	MBBS, MD (Medic...	₹800.00	Room 104	Daily (Mon-Sun)	+91-98765-432...
DOC-202	Dr. Ananya Iyer	Neurology	MBBS, M.Ch (Neu...	₹1500.00	Room 102	Tue, Thu, Sat	+91-98765-432...
DOC-204	Dr. Priya Deshmu...	Pediatrics	MD (Pediatrics)	₹900.00	Room 205	Daily (Mon-Sat)	+91-98765-432...
DOC-201	Dr. Rajesh Sharma	Cardiology	MD, DM (Cardiol...	₹1200.00	Room 101	Mon, Wed, Fri	+91-98765-432...
DOC-206	Dr. Sunita Sen	Dermatology	MD (Dermatology)	₹1100.00	Room 302	Wed, Fri, Sat	+91-98765-432...
DOC-203	Dr. Vikram Malho...	Orthopedics	MS (Orthopedics)	₹1000.00	Room 201	Mon, Tue, Thu	+91-98765-432...

Figure 3: Doctor Directory and Specialization Filter

6.4 Appointment Scheduling (Queue) Module

Appointments are managed using a strict LinkedList Queue. Patients are assigned to doctors for specific time slots. Receptionists can mark appointments as 'Completed' or 'Cancelled', which instantly dequeues the item.

CarePulse
Hospital Management System

Dashboard

Patients

Doctors

Appointments

Medical Records

Billing

Sorted Records

Patient Appointment Management (LinkedList Queue)
Schedule consultations, maintain FIFO queue order, and update visit statuses

Schedule Consultation

Select Patient *:Aarav Gupta (PAT...

Assign Doctor *:Dr. Amit Patel - ...

Time Slot:09:00 AM

Days from Today:0

Visit Reason / Chief C...

Book Appointment (Enqueue)

Filter Status:All Statuses

Mark CompletedCancel AppointmentRefresh Queue

Queue #	Apt ID	Patient Na...	Doctor	Date	Slot	Status	Reason
#1	APT-5001	Aarav Gu...	Dr. Amit ...	2026-09...	10:00 AM	Completed	Routine ...
#2	APT-5002	Rohan M...	Dr. Rajes...	2026-09...	11:30 AM	Scheduled	Chest tig...
#3	APT-5003	Zara Kha...	Dr. Priya ...	2026-09...	02:30 PM	Scheduled	Seasonal...
#4	APT-5004	Meera Ku...	Dr. Anan...	2026-09...	09:30 AM	Scheduled	Chronic ...
#5	APT-5005	Devendra...	Dr. Vikra...	2026-09...	03:00 PM	Completed	Severe k...

Figure 4: Appointment Booking and Queue Management

6.5 Medical Records Module

Doctors use this module to append clinical history, diagnoses, and prescriptions to a patient's file. The records are tied to the Patient ID and can be retrieved instantly for future consultations.

CarePulse
Hospital Management System

Dashboard

Patients

Doctors

Appointments

Medical Records

Billing

Sorted Records

Patient Medical Records & Clinical History (LinkedList)
Record diagnoses, prescriptions, lab investigations, and maintain patient health files

New Clinical Consultation Entry

Patient *:Aarav Gupta (PAT-1001)

Attending Doctor...Dr. Amit Patel

Chief Symptoms *:

Clinical Diagnosi...

Rx / Prescription:

Lab Tests Advise...None

Doctor's Notes:

Save Medical Record

Filter by Patient:All Patients

View Full Prescription

Record ID	Patient ID	Attending Doct...	Visit Date	Diagnosis	Prescription Su...
MED-7001	PAT-1001	Dr. Amit Patel	2026-09-06	Viral pharyng...	Paracetamol ...
MED-7002	PAT-1003	Dr. Rajesh Sh...	2026-09-11	Stage 1 Hype...	Amlodipine 5...
MED-7003	PAT-1005	Dr. Vikram M...	2026-09-18	Osteoarthritis...	Glucosamine ...

Figure 5: Medical Records and Clinical Entry Form

6.6 Billing & Invoicing Module

Financial transactions are managed here. The module calculates Consultation Fees, Lab Fees, and Pharmacy charges, applies tax rates, and generates a final payable total. Invoices are stored in a HashMap.

CarePulse
Hospital Management System

Dashboard

Patients

Doctors

Appointments

Medical Records

Billing

Sorted Records

Hospital Billing & Financial Invoicing (HashMap Lookup)
Generate itemized hospital bills, process patient payments, and track revenue

Generate Patient Invoice

Patient *:
Aarav Gupta (PAT-1...

Doctor Consultatio...
1000.00

Lab / Diagnostic (₹):
0.00

Pharmacy / Meds (...
0.00

Room / Ward Char...
0.00

Tax Rate (%):
5.0

Discount (₹):
0.00

Net Payable Total:
₹1050.00

Payment Method:
Cash

Payment Status:
Pending

Generate Invoice

Filter Status: All Bills

Mark as Paid

Print / View Invoice

Bill ID	Patient Na...	Consult Fee	Lab Fee	Meds Fee	Total (₹)	Status	Date
INV-9002	Rohan M...	₹1200.00	₹950.00	₹450.00	₹2730.00	Pending	2026-09...
INV-9003	Devendra...	₹1000.00	₹600.00	₹400.00	₹1950.00	Paid	2026-09...
INV-9001	Aarav Gu...	₹800.00	₹350.00	₹250.00	₹1370.00	Paid	2026-09...

Figure 6: Financial Billing and Invoice Generation

6.7 Sorted Records Module

This reporting module leverages the underlying TreeMap. It allows administrators to view the entire patient directory naturally sorted by alphabetical order without requiring any runtime sorting algorithms (like Bubble Sort or Merge Sort), guaranteeing massive performance boosts.

CarePulse
Hospital Management System

Dashboard
Patients
Doctors
Appointments
Medical Records
Billing
Sorted Records

Sorted Patient Records

Explore and filter patient records using custom sorting

Select Sort Criterion: Alphabetical by Patient Name ⌵ Re-apply Sort

Sorted alphabetically by Patient Name.

Index	Patient ID	Full Name	Age	Gender	Blood Group	Contact Number	Reg. Date	BP
1	PAT-1001	Aarav Gupta	29 yrs	Male	O+	+91-91234-5...	2026-09-06	120/80
2	PAT-1005	Devendra Joshi	68 yrs	Male	O-	+91-91234-5...	2026-09-16	135/88
3	PAT-1007	Karan Singhanian	24 yrs	Male	B-	+91-91234-5...	2026-09-20	122/80
4	PAT-1002	Meera Kulkarni	42 yrs	Female	B+	+91-91234-5...	2026-09-09	130/85
5	PAT-1006	Pooja Verma	34 yrs	Female	A-	+91-91234-5...	2026-09-19	118/78
6	PAT-1003	Rohan Mehta	56 yrs	Male	A+	+91-91234-5...	2026-09-11	140/90
7	PAT-1004	Zara Khan	8 yrs	Female	AB+	+91-91234-5...	2026-09-13	110/70

Figure 7: TreeMap-powered Alphabetical Patient Directory

7. Conclusion

The CarePulse Hospital Management System successfully proves that highly complex, data-intensive applications can be engineered securely and efficiently using core Java constructs. By strictly adhering to the MVC architecture and deliberately selecting optimal data structures (HashMaps for speed, TreeMaps for order, LinkedLists for queues), the system achieves zero-latency performance.

Future enhancements could include the integration of a persistent SQL database (like MySQL or PostgreSQL) to permanently save the data structures to disk, as well as role-based authentication (Admin, Doctor, Receptionist) to secure the UI panels.